

光栅化 · 直线

现代图形管线很多都是基于光栅化的。光栅化过程中，如何光栅化两点 $p_1(x_1, y_1)$ 和 $p_2(x_2, y_2)$ 之间的直线？这篇笔记给出了三种经典的算法原理讲解和代码实现（代码实现基于 [EasyX 库](#)）。其中——

- DDA 算法使用了浮点数乘法
- 中点算法仅使用了整形乘法
- Bresenham 算法仅仅使用了正数加减法

这些算法虽然已经在 GPU 硬件上高效实现了，但是可以推广到其他地方。最常见的是在《我的世界》《泰拉瑞亚》这种将世界网格化的游戏中实现一些效果。

一、Digital Differential Analyzer (DDA)

DDA 基于直线表达式

$$y = kx + b$$

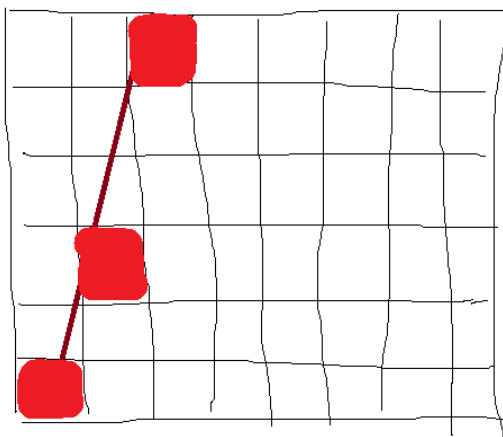
首先确定端点 p_1 和 p_2 所在的像素 P_1 和 P_2 ，将坐标取整即可。

然后按照 1 的步长来递增 x ，然后迭代求 y ($y_{i+1} = y_i + k$)。

```
y = y1
x = x1
while x != x2:
    drawPixel(x, y)
    y += k
    x += 1
```

Python

这样有一个问题：如果 y 的跨度太大（ k 太大），线段就会断掉——



解决方法是看是 $|dx|$ 大还是 $|dy|$ 大。如果 $|dx|$ 大的话说明横向变化更剧烈，选定横向为递增方向；如果 $|dy|$ 更大的话说明纵向变化更剧烈，选定纵向为递增方向。

▼ DDA 光栅化直线

C++

```

void DDALineRenderer::draw(const DPoint& p1, const DPoint& p2)
{
    int x1 = (int)p1.x;
    int y1 = (int)p1.y;
    int x2 = (int)p2.x;
    int y2 = (int)p2.y;
    int dx = x2 - x1;
    int dy = y2 - y1;
    if (dx == 0) //处理斜率无限大的特殊情况
    {
        int sign = (y2 - y1) > 0 ? 1 : -1;
        for (int y = y1; y != y2; y += sign)
        {
            putpixel(x1, y, WHITE);
        }
    }
    else if (dy == 0) //处理斜率无限大的特殊情况
    {
        int sign = (x2 - x1) > 0 ? 1 : -1;
        for (int x = x1; x != x2; x += sign)
        {
            putpixel(x, y1, WHITE);
        }
    }
    else
    {
        if (abs(dx) > abs(dy)) //横向作为递增方向
        {
            double k = 1.0 * dy / dx;
            int sign = (x2 - x1 > 0) ? 1 : -1;
            double y = y1;
            for (int x = x1; x != (x2+sign); x += sign)
            {
                putpixel(x, (int)y, WHITE);
                if (sign > 0)
                {
                    y += k;
                }
                else y -= k;
            }
        }
        else //纵向作为递增方向
        {
            double k = 1.0 * dx / dy;
            int sign = (y2 - y1 > 0) ? 1 : -1;
            double x = x1;
            for (int y = y1; y != (y2+sign); y += sign)
            {
                putpixel((int)x, y, WHITE);
                if (sign > 0)
                {

```

```

        x += k;
    }
    else x -= k;
}
}
}
}
}

```

二、中点画线法

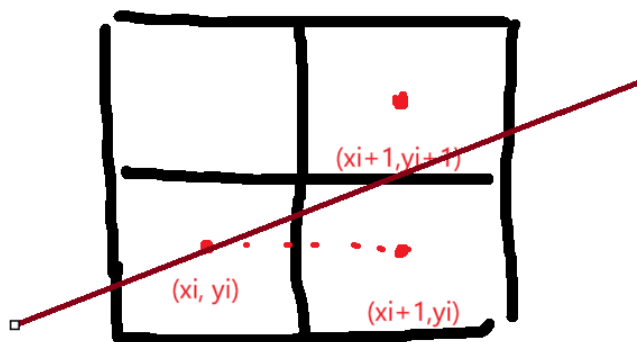
2.1 原理

中点画线法基于直线表达式

$$Ax + By + C = 0$$

从 P_1 到 P_2 迭代绘制。

简单起见，我们假设直线的斜率在(0,1)范围内。在这种情况下，假设当前像素为 (x_i, y_i) ，那么下一个像素要么是 $p'(x_i + 1, y_i)$ ，要么是 $p''(x_i + 1, y_i + 1)$ ——



具体取哪个值我们将 p' 和 p'' 的中点 $(x_i + 1, y_i + \frac{1}{2})$ 代入表达式判别在直线上面 ($d > 0$, 取 p'') 还是下面 ($d < 0$, 取 p') ——

$$d = A(x + 1) + B(y + \frac{1}{2}) + C$$

2.2 初始化&迭代

2.1.1 初始状态

初始状态下我们计算 d 值——

$$d = A(x_0 + 1) + B(y_0 + \frac{1}{2}) + C$$

$$= Ax_0 + By_0 + C + A + \frac{1}{2}B$$

$$= 0 + A + \frac{1}{2}B$$

$$= A + \frac{1}{2}B$$

2.2.2 迭代

在已知上一个点 (x_i, y_i) 的情况下，迭代计算 d_{new} 。

- d_{old} 表达式——

$$d_{old} = A(x_i + 1) + B(y_i + \frac{1}{2}) + C$$

如果 $d_{old} > 0$ ，则当前点坐标为 $(x_i + 1, y_i + 1)$ ，决定下一个点位置的 d_{new} 计算表达式为——

$$d_{new} = A(x_i + 2) + B(y_i + (1 + \frac{1}{2})) + C$$

$$= d_{old} + A + B \text{ ——增量 } \Delta d = A + B。$$

如果 $d_{old} < 0$ ，则当前点坐标为 $(x_i + 1, y_i)$ ，决定下一个点位置的 d_{new} 计算表达式为——

$$d_{new} = A(x_i + 2) + B(y_i + \frac{1}{2})$$

$$= d_{old} + A \text{ ——增量 } \Delta d = A。$$

2.2.3 优化

注意到初始情况下的 d 值为 $A + \frac{1}{2}B$ ，出现了浮点数乘法。因为我们只关注 d 的符号，而添加一个正数因子并不会改变它的符号。我们取 $2d$ ，这样的话 $2d = 2A + B$ ，就消除了小数 $\frac{1}{2}$ 。后续的迭代得到的 d 也跟着 $\times 2$ 。这样， d_{new} 的迭代增量修改为 $2(A + B)$ 或者 $2A$ 。

2.3 示例

假设我们要从点 $p_0(0, 0)$ 画线到 $p_1(3, 1)$ 。

初始化

我们需要计算 A 、 B （观察到 C 不会在迭代的过程中用到，所以我们不用计算 C ）和 d 。

根据直线两点式方程→一般方程，我们可以得到

$$A = y_1 - y_0 = 1$$

$$B = x_0 - x_1 = -3$$

$$d = 2A + B = -1$$

我们在 p_0 点填入颜色。令 $d_0 = d$ ， $x = x_0 = 0$ ， $y = y_0 = 0$ 开始迭代。

迭代

迭代 1: 由于 $d_0 < 0$ ，因此我们在 $(x + 1, y)$ （即 $(1, 0)$ ）像素填入颜色。计算 $d_1 = d_0 + 2A = 1$ 。更新 $x = x + 1 = 1$ ， $y = y = 0$ 。

迭代 2: 由于 $d_1 > 0$ ，因此我们在 $(x + 1, y + 1)$ （即 $(2, 1)$ ）像素填入颜色。计算 $d_2 = d_1 + 2(A + B) = -3$ 。更新 $x = x + 1$ ， $y = y + 1$ 。

迭代 3: 由于 $d_2 < 0$ ，因此我们在 $(x + 1, y)$ （即 $(3, 1)$ ）像素填入颜色。计算 $d_3 = d_2 + 2A = -1$ 。更新 $x = x + 1$ ， $y = y$ 。

完毕。我们绘制点的顺序为： $(0, 0) \rightarrow (1, 0) \rightarrow (2, 1) \rightarrow (3, 1)$ 。

2.4 推广到整个平面

我们上面的推理是基于直线斜率处于 $(0, 1)$ 区间内的特殊情况。我们现在将其推广到整个平面——

给定平面直角坐标系内两个顶点 $p_0(x_0, y_0)$ 和 $p_1(x_1, y_1)$ ，光栅化两点之间的直线。

我们首先要考虑的是将 x 轴设为递增主轴还是将 y 轴设为递增主轴。根据上面的讨论，如果 $|\Delta x| > |\Delta y|$ ，说明直线在 x 轴上的变化更加剧烈，我们就将 x 轴设置为主轴；反之将 y 轴设置为主轴。

几何上选定 x 轴和选定 y 轴是对称的，我们在这里详细给出将 x 作为主轴的情形。将 y 轴作为主轴的情形是类似推导的。

2.4.1 x 轴设定为主轴

将 x 轴设定为主轴的情况下， x 轴恒定每次迭代都会递增；我们关心的是 y 轴上是否需要递增。即当前点为 (x, y) 的情况下，下一个点是 $(x + 1, y)$ 还是 $(x + 1, y + 1)$ 。为此我们需要判断中点 $(x + 1, y + \frac{1}{2})$ 在直线上方还是下方。

对于直线的标准方程

$$d = Ax + By + C = 0$$

假设某个点 $P(x_p, y_p)$ 在直线上，则满足代入之后结果为 0。

那么对于在直线 $x = x_p$ 上的另一个点 $Q(x_q, y_q)$ 我们如何判断 Q 在点 P 上面还是下面？

2.4.1.1 判断点在直线上方/下方

情况 1: $d > 0$

将 Q 点坐标代入后经过简单移项，我们得到

$$By_q > -(Ax_q + C)$$

这个时候如果 $B > 0$ ，那么就有 $y_q > -\frac{Ax_q + C}{B}$ 。这意味着什么？因为 $x_q = x_p$ ，原式即

$$y_q > -\frac{Ax_p + C}{B}$$

—— Q 点在 P 点上面。反之，如果 $B < 0$ ，那么 Q 点在 P 点下面。

情况 2: $d < 0$

类似的，如果 $B < 0$ ，就有

$$y_q > -\frac{Ax_p + C}{B}$$

—— Q 点在 P 点上面。反之，如果 $B > 0$ ，那么 Q 点在 P 点下面。

总结

- $d \cdot B > 0$ ：点在直线上方
- $d \cdot B < 0$ ：点在直线下方

2.4.1.2 x 轴位移和 y 轴位移

现在既然推广到全平面，那么 x 轴和 y 轴上的位移肯定不是 1。我们定义

$$s_x = \text{sign}(x_1 - x_0)$$

$$s_y = \text{sign}(y_1 - y_0)$$

其中 sign 返回数值的符号，正数返回 1，0 返回 0，负数返回 -1 。

因为我们涂色的方向是从 (x_0, y_0) 向 (x_1, y_1) ，因此 x 轴和 y 轴上的单位迭代增量就是 s_x 和 s_y 。

2.4.1.3 新条件下迭代推理

$$A = y_1 - y_0$$

$$B = x_0 - x_1$$

$$d_0 = A(x_0 + s_x) + B(y_0 + \frac{1}{2} \cdot s_y) + C$$

$$= s_x \cdot A + \frac{1}{2} \cdot s_y \cdot B$$

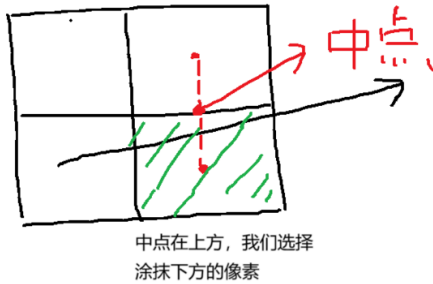
去除小数： $2d_0 = 2 \cdot s_x \cdot A + s_y \cdot B$

- 如果 $d_0 \cdot B > 0$

说明测试点 $P_{middle}(x_0 + s_x, y_0 + \frac{1}{2} \cdot s_y)$ 在直线上方.

- (1) $s_y > 0$

这种情况下, y 轴方向是递增的, 而中间点在直线上方——



很显然, 这种情况下我们应该选择下方的像素而不是上方的。选择下方的像素意味着我们 y 轴上不进行递增, 保持不变。因此新的像素点为 $(x_0 + s_x, y_0)$ 。下一个测试的中心点为 $(x_0 + 2 \cdot s_x, y_0 + \frac{1}{2} \cdot s_y)$ 计算新的 d_1 ——

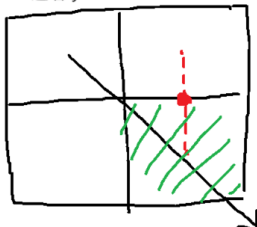
$$2d_1 = 2 \cdot [A \cdot (x_0 + 2 \cdot s_x) + B \cdot (y_0 + \frac{1}{2} \cdot s_y) + C]$$

$$= 2d_0 + 2 \cdot s_x \cdot A \quad \text{——增量为 } 2 \cdot s_x \cdot A$$

- (2) $s_y < 0$

这种情况下说明直线在 y 轴上是向下的。而中间点在直线上方——

直线向下。中点在上的情况下, 需要
递增 s_y



上图中很显然直线足够倾斜才导致中点在直线上方。因此我们需要在 y 轴上向下移动一个单位 (即 s_y)。新的像素点为 $(x_0 + s_x, y_0 + s_y)$ 。新的中心点为 $(x_0 + 2 \cdot s_x, y_0 + 1.5 \cdot s_y)$ 计算新的 d_1 ——

$$2d_1 = 2 \cdot [A(x_0 + 2 \cdot s_x + B \cdot (y_0 + (\frac{1}{2} + 1) \cdot s_y) + C)]$$

$$= 2d_0 + 2 \cdot (s_x \cdot A + s_y \cdot B) \quad \text{——增量为 } 2 \cdot (s_x \cdot A + s_y \cdot B)$$

- 如果 $d_0 \cdot B < 0$

说明测试点在直线的下方。那么现在的情况和上面 $d_0 \cdot B > 0$ 情况刚好相反。

◦ $s_y > 0$

新的像素点为 $(x_0 + s_x, y_0 + s_y)$ ；新的中心点为 $(x_0 + 2 \cdot s_x, y_0 + (\frac{1}{2} + 1) \cdot s_x)$ 。

$2d_1 = 2d_0 + 2 \cdot (s_x \cdot A + s_y \cdot B)$ ——增量为 $2 \cdot (s_x \cdot A + s_y \cdot B)$

◦ $s_y < 0$

新的像素点为 $(x_0 + s_x, y_0)$ ，新的中心点为 $(x_0 + 2 \cdot s_x, y_0 + \frac{1}{2} \cdot s_x)$ 。

$2d_1 = 2d_0 + 2 \cdot s_x \cdot A$ ——增量为 $2 \cdot s_x \cdot A$

2.4.1.4 总结

我们根据上面所有条件总结下来，可以得到下面的结果——

- 如果 $d \cdot B \cdot s_y < 0$ ：则 y 轴上需要递增 s_y 。
- 如果 $d \cdot B \cdot s_y > 0$ ：则 y 轴上不需要递增 s_y

2.4.1 y 轴设定为主轴

和将 x 轴设定为主轴是对称的。这里直接给出结果——

- 如果 $d \cdot A \cdot s_x < 0$ ： x 轴上需要递增 s_x
- 如果 $d \cdot A \cdot s_x > 0$ ： x 轴上不需要递增 s_x

2.5 代码

经过上面的推导之后，我们很容易写出在全平面用中点法光栅化直线的代码——

```
C++

void MiddlePointLineRenderer::draw(const DPoint& p1, const DPoint& p2)
{
    int x0 = (int)p1.x;
    int y0 = (int)p1.y;
    int x1 = (int)p2.x;
    int y1 = (int)p2.y;

    int dx = x1 - x0;
    int dy = y1 - y0;

    int sx = sign(dx);
    int sy = sign(dy);

    if (dx == 0) // 处理斜率特殊情况
    {
        int y = y0;
        while (y != y0)
        {
            putpixel(x0, y, WHITE);
        }
    }
}
```



```

        y = y + sy;
    }
}
if (dy == 0)
{
    int x = x0;
    while (x != x1)
    {
        putpixel(x, y0, WHITE);
        x = x + sx;
    }
}
int A = y1 - y0;
int AA = A + A;
int B = x0 - x1;
int BB = B + B;

int x = x0, y = y0;
if (abs(dx) >= abs(dy)) //x轴设定为主轴
{
    int d = sx * AA + sy * B; //2 * A + B
    putpixel(x, y, WHITE);
    while (x != x1)
    {
        if (d * B * sy < 0)
        {
            putpixel(x + sx, y + sy, WHITE);
            y = y + sy;
            d = d + sx * AA + sy * BB; // d_1 = d_0 + 2 * (A + B)
        }
        else
        {
            putpixel(x + sx, y, WHITE);
            d = d + sx * AA; // d_1 = d_0 + 2 * A
        }
        x = x + sx;
    }
}
else //y轴设定为主轴
{
    int d = sx * A + sy * BB; // A + 2 * B
    putpixel(x, y, WHITE);
    while (y != y1)
    {
        if (d * A * sx < 0)
        {
            putpixel(x + sx, y + sy, WHITE);
            x = x + sx;
            d = d + sx * AA + sy * BB; //d_1 = d_0 + 2 * B
        }
        else
        {

```

```

    putpixel(x, y + sy, WHITE);
    d = d + sy * BB;
}
y = y + sy;
}
}

```

三、Bresenham 算法

Bresenham 算法是基于直线的斜截式来进行迭代的——

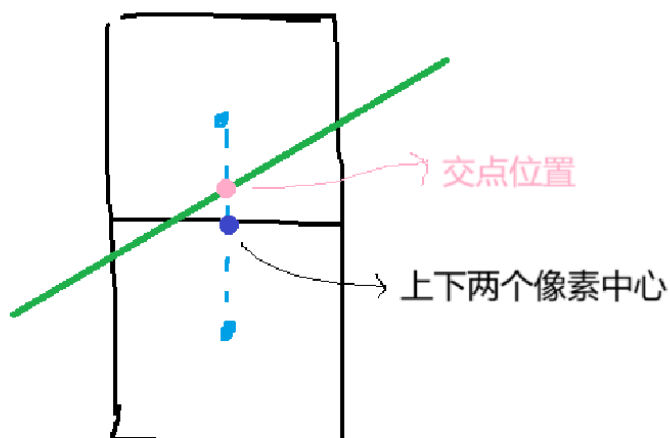
$$y = kx + b$$

为了便于理解我们先从斜率处于 $(0, 1)$ 条件下推导，然后推广到整个平面。

3.1 斜率 $(0, 1)$ 条件下

这种情况下， x 轴是递增主轴，并且每次迭代都让 x 递增 1。

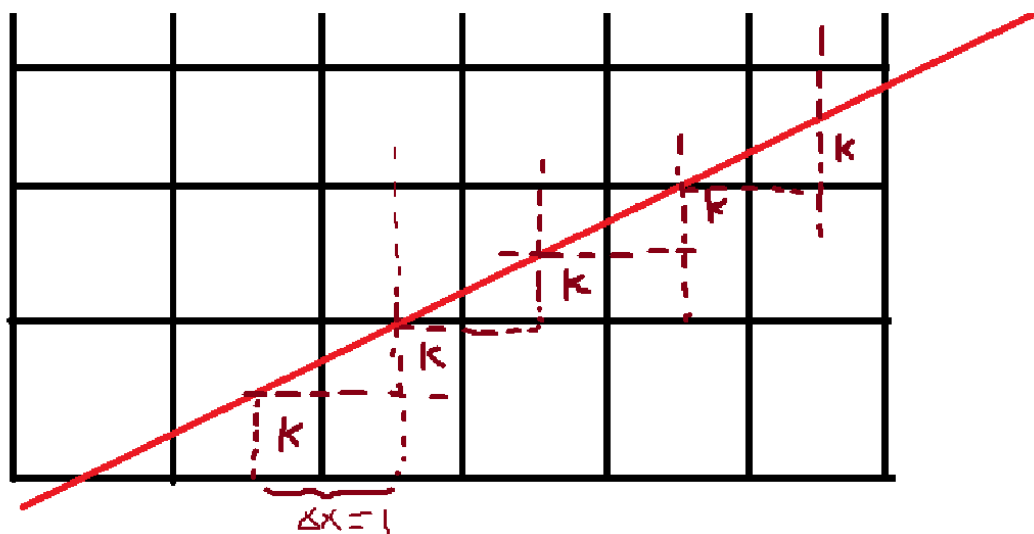
Bresenham 算法的核心是：判断第 n 个像素中心点所在的垂直线 $x = x_n$ 与光栅化直线的交点是“靠上”还是“靠下”。如果是“靠上”则涂抹上方的像素，如果“靠下”则涂抹下方的像素。“靠上”还是“靠下”的判断方法是光栅化直线与 $x = x_n$ 交点和其所在附近上下两个像素中间的相对位置——



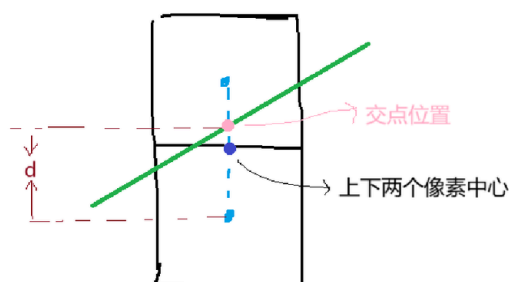
交点“靠上”，因此涂抹上方的像素

上图中，直线与 $x = x_n$ 交点在两个像素中央的上方，因此我们涂抹的是上面的像素而不是下面的。

我们可以每次迭代都计算一次像素中央和交点的相对位置，但这不是最好的方法。我们这里采用的方法是初始化一个 $d = 0$ ，然后每次 x 递增则跟着递增 $d = d + k$ 。下面的图显示了这种做法的原理——



我们初始化 $d = 0$ ，并且每次 d 递增之后如果 > 1 （说明交点到了下一行像素）则让 $d = d - 1$ 。这样能够始终保证 d 代表的是光栅化直线与 $x = x_n$ 的交点到它下方的像素中心的距离——



交点“靠上”，因此涂抹上方的像素

通过判断 $d > 0.5$ 来判断交点是更加靠近下方像素还是上方像素。

但是判断式中出现了浮点数 0.5。为了避免算法中出现浮点数操作，我们通过引入一个变量 e 来消除它同时将判断式改为和 0 比较——

$$e = d - 0.5, 2e = 2d - 1$$

这样 $e > 0$ 等价于 $d > 0.5$ ，更靠近上方； $e < 0$ 等价于 $d < 0.5$ ，更靠近下方。

另外，我们需要每次将 d 递增 k ，而 $k = \frac{dy}{dx}$ ，出现了除法。我们继续修正判别表达式为——

$$a = 2e\Delta x = 2(d - 0.5)\Delta x, \Delta x = x_1 - x_0$$

这样修正之后因为 2 和 Δx 都是正数，因此不改变和 0 比较的判别结果。经过这样修正之后有——

- a 的初始值为 $2(0 - 0.5)\Delta x = -\Delta x$
- 每次 $d > 1$ 的时候都要进行修正 $d = d - 1$ 。对应的，每次 $a > 2(1 - 0.5)\Delta x = \Delta x$ 的时候都要修正一个 $2\Delta x$ ，即 $a = a - 2\Delta x$ 。
- 每次 d 递增 k ， a 递增量为——

$$2(d - 0.5 + k)\Delta x$$

$$= (2d - 1 + 2\frac{\Delta y}{\Delta x})\Delta x$$

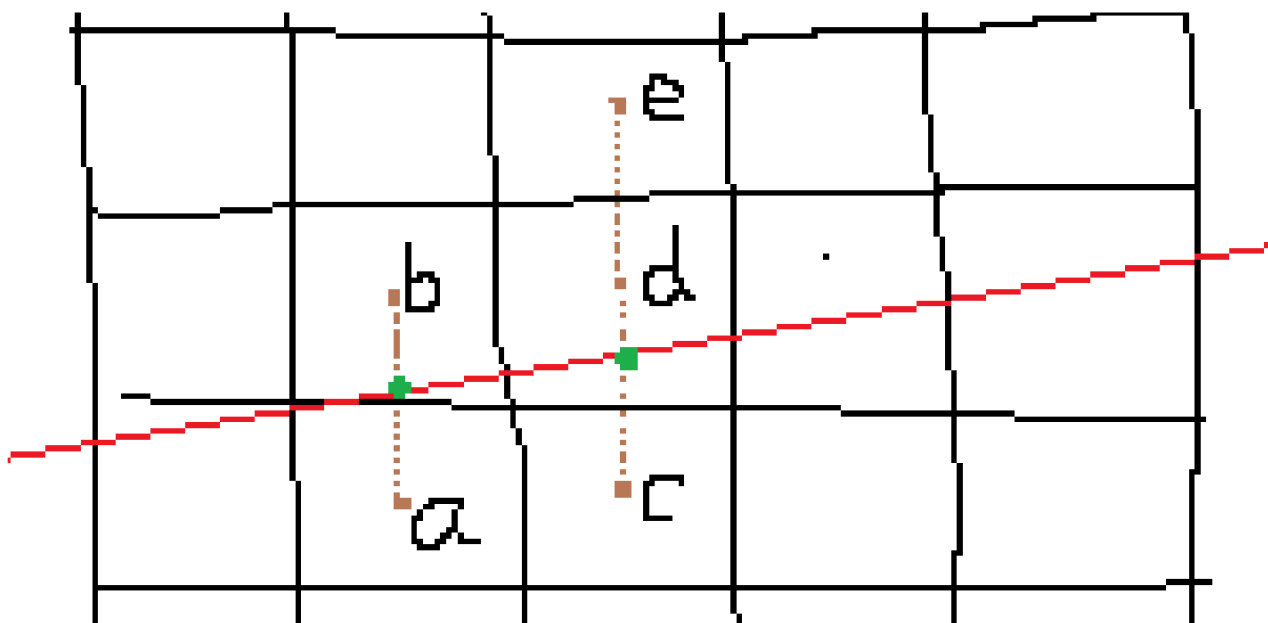
$= a + 2\Delta y$ —— $\Delta a = 2\Delta y$ ，即 $a = a + 2\Delta y$ 。可以看见为什么要引入 Δx ——消除了浮点数 k 中的分母 Δx 。

更新 y

这里有一个细节需要注意，就是我们什么时候更新我们的 $y = y + 1$ 。

我们在绘制的过程中，假设上一个像素的坐标是 (x, y) ，如果光栅化直线与 $x = x_n$ 直线相交的结果更加靠近上方的像素，我们就应该在 $(x + 1, y + 1)$ 的位置绘制。但是我们这个时候需要更新 y 吗？

答案是否定的，看下面这张图——



可以看见连续两次交点都更靠近上方。如果第一次交点靠近上方的时候我们更新 $y = y + 1$ ，那么第二次我们就是拿交点和 d 、 e 的中心比较而不是 c 、 d ；这样一来 d 的值变成了负数（记住 d 代表的是交点到上下两个像素中下方像素的中心之间的距离），出现了异常。

那我们什么时候更新 y 呢？答案很简单：当 $d > 1$ 的时候。 $d > 1$ 意味着交点已经向上跨完整的一行，因此需要将 y 更新为 $y + 1$ 。其他情况下即使 $d > 0$ 我们也只是在 $(x + 1, y + 1)$ 像素处绘制，而不是将 y 递增。

3.2 示例

我们以光栅化 $p_0(0, 0)$ ， $p_1(3, 1)$ 为例来实现上面的过程。

初始条件

$$\Delta x = 3 - 0 = 3$$

$$\Delta y = 1 - 0 = 1$$

$$a = -\Delta x = -3$$

在像素位置 $(0, 0)$ 涂抹颜色。

迭代 1

$a = a + 2\Delta y = -3 + 2 = -1$ ，因为 $a \leq 0$ ，在当前行像素—— $(x + 1, y) = (1, 0)$ 像素点涂上颜色。

a 满足 $< \Delta x$ ，保持不变；不递增 y 。

迭代 2

$a = a + 2\Delta y = -1 + 2 = 1$ ，因为 $a \geq 0$ ，我们在上面一行像素—— $(x + 2, y + 1) = (2, 1)$ 涂抹颜色。

a 满足 $< \Delta x$ ，保持不变；不递增 y 。

迭代 3

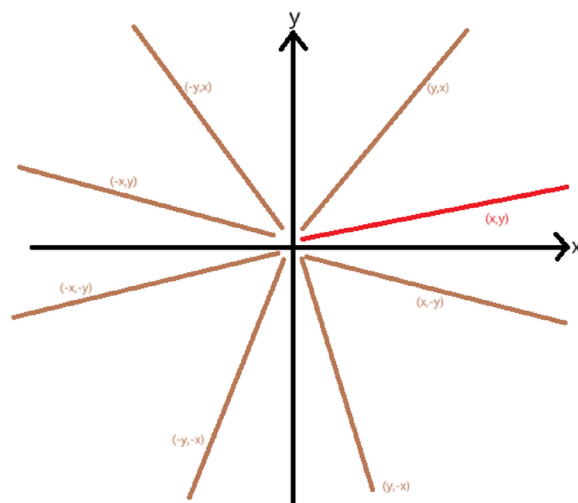
$a = a + 2\Delta y = 1 + 2 = 3 = \Delta x$ 。因为 $a \leq 0$ ，在 $(x + 3, y) = (3, 1)$ 像素上涂抹颜色。

a 不满足 $< \Delta x$ ，更新 a 为 $a = a - 2 \cdot \Delta x = -3$ ，同时递增 $y = y + 1 = 1$ 。

3.3 推广到平面

上面中点算法中我们推广到平面采用的方法是简单的数学推导。这里我们采用一种新的方法来实现——镜像对称。

下面图中给出了其他七个方向的直线和 $k \in (0, 1)$ 直线的坐标对称关系——



基于这张图，我们可以很方便地通过对称的方法来将其他斜率条件下的直线对称到 $k \in (0, 1)$ 范围内。

下面是代码实现——

```
void BresenhamLineRenderer::draw(const DPoint& p1, const DPoint& p2)
{
```

C++

```

int x0{ 0 }, y0{ 0 }, x1{ 0 }, y1{ 0 };
//保证(x0,y0)在左边，可以极大简化条件
if (p1.x <= p2.x)
{
    x0 = (int)p1.x;
    y0 = (int)p1.y;
    x1 = (int)p2.x;
    y1 = (int)p2.y;
}
else
{
    x0 = (int)p2.x;
    y0 = (int)p2.y;
    x1 = (int)p1.x;
    y1 = (int)p1.y;
}

int dx = x1 - x0;
int dy = y1 - y0;

int sx = sign(dx);
int sy = sign(dy);

//处理斜率特殊情况

if (!sx)
{
    int y = y0;
    while (y != (y1 + sy))
    {
        putpixel(x0, y, WHITE);
        y = y + sy;
    }
    return;
}

if (!sy)
{
    int x = x0;
    while (x != (x1 + sx))
    {
        putpixel(x, y0, WHITE);
        x = x + sx;
    }
    return;
}

int _x0, _y0, _x1, _y1;

```

```

//增长-x主轴，增长-y主轴，降低-x主轴，降低-y主轴
enum LINE_CASE { INC_X = 0, INC_Y, DEC_X, DEC_Y } line_case;

//我们将线的起初坐标、终点坐标对称到斜率小于1的标准情形下
if (dy > 0)
{
    if (dx > dy)
    {
        _x0 = x0; _y0 = y0;
        _x1 = x1; _y1 = y1;
        line_case = INC_X;
    }
    else
    {
        _x0 = y0; _y0 = x0;
        _x1 = y1; _y1 = x1;
        line_case = INC_Y;
    }
}
else
{
    if (dx > -dy)
    {
        _x0 = x0; _y0 = -y0;
        _x1 = x1; _y1 = -y1;
        line_case = DEC_X;
    }
    else
    {
        _x0 = -y0; _y0 = x0;
        _x1 = -y1; _y1 = x1;
        line_case = DEC_Y;
    }
}

int x = _x0;
int y = _y0;
int _dx = _x1 - _x0;
int _dy = _y1 - _y0;
int a = -_dx;

putpixel(x0, y0, WHITE);
while (x < (_x1 + sx)) //我们上面保证了sx = 1
{
    int draw_x = x + 1, draw_y = y;

    a = a + _dy + _dy;
    if (a >= _dx)
    {
        a = a - _dx - _dx;
        y = y + 1;
    }
}

```

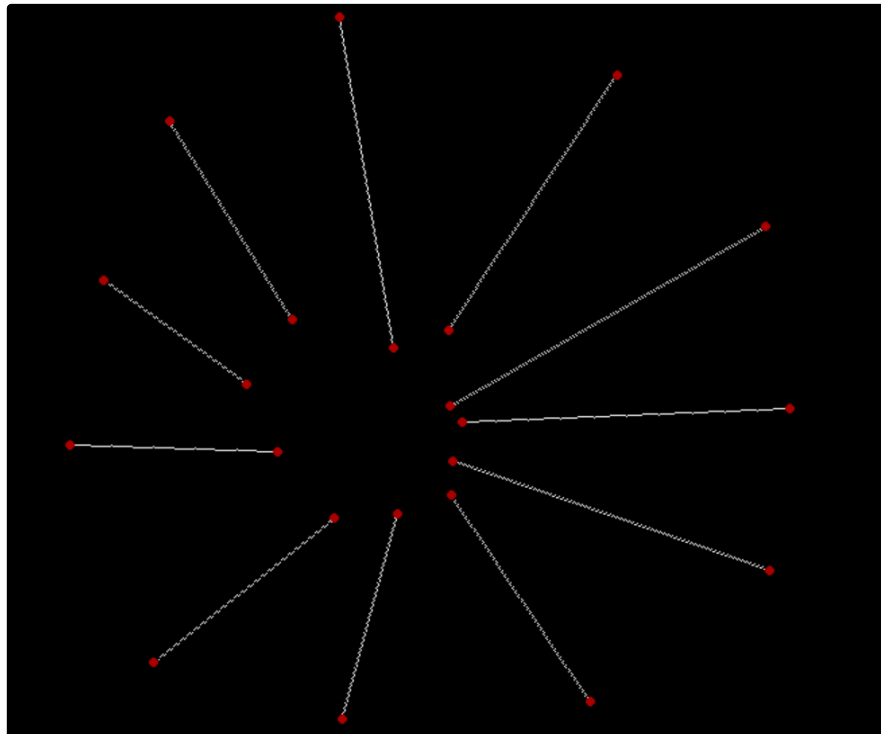
```

}
if (a <= 0)
{}
else
{
    draw_y = y + 1;
}
switch (line_case)
{
    case INC_X:
        putpixel(draw_x, draw_y, WHITE);
        break;
    case INC_Y:
        putpixel(draw_y, draw_x, WHITE);
        break;
    case DEC_X:
        putpixel(draw_x, -draw_y, WHITE);
        break;
    case DEC_Y:
        putpixel(draw_y, -draw_x, WHITE);
        break;
    default: break;
}
x = x + 1;
}

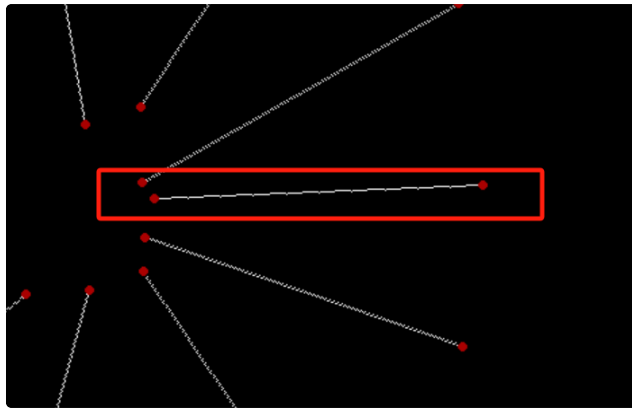
```

3.4 边界问题

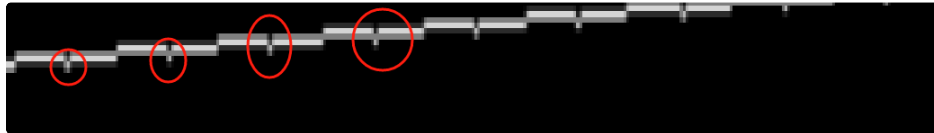
下面是使用上述代码绘制的结果（左上角是原点， x 轴向右， y 轴向下）——



会发现一个非常奇怪的现象：上图中的横向朝右的那根线——



y 值出现了很奇怪的“抖动”。仔细观察可以发现在两段比较长的线之间 y 值突然突出一个像素。我们放大 能看得更清楚——



这是为什么呢？关键在于这段更新 a 的代码——

```
C++
a = a + _dy + _dy;

//判断并且更新a
if (a >= _dx)
{
    a = a - _dx - _dx;
    y = y + 1;
}

//根据a的值来决定绘制像素
if (a <= 0)
{}
else
{
    draw_y = y + 1;
}
```

这里的做法是：我们先判断($a \geq _dx$)并更新了 a ，然后再根据更新后的 a 的值小于等于 0 还是大于 0 来决定是在 $(x + 1, y)$ 还是 $(x + 1, y + 1)$ 涂颜色。

这里很显然顺序错了。现在假设 a 的值 > 0 ，一直选择的是 $(x_i + 1, y_i + 1)$ 像素；但是当 a 经过

```
C++
a = a + _dy + dy
```

更新之后如果 $a = _dx$ 成立，这样 a 就会被更新——

```
Python
a = a - _dx - _dx
```

由于这里的操作本质是当 $d == 1$ 的时候进行 $d = d - 1 = 0$ ，因此 a 会被更新为 0。结果 $a \leq 0$ 因为 $a = 0$ 而成立，进而这一次的绘制选择了 $(x_i + 1, y)$ 像素，这就导致了原本一直在 $(x_i + 1, y + 1)$ 像素绘制的中间突然出现了一次 $(x_i + 1, y_i)$ 处像素的绘制——这就导致了上图中有像素突出的现象。

知道问题之后下面给出修复方法——

替换位置

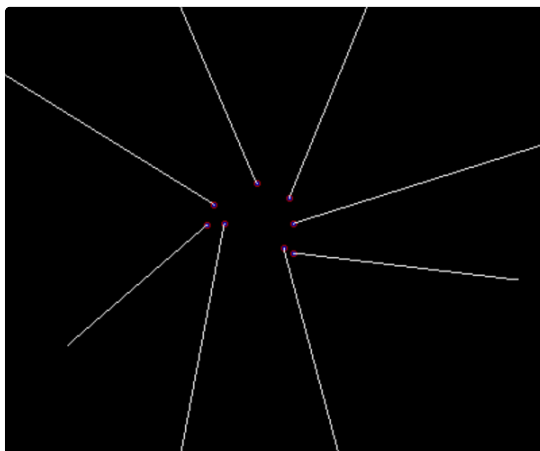
很显然，既然顺序错了，那么就先根据 a 的值是否大于 0 来决定绘制的像素，然后再判断并且更新 a

```
JavaScript

a = a + _dy + _dy;

//根据a的值来决定绘制像素
if (a <= 0)
{}
else
{
    draw_y = y + 1;
}
//判断并且更新a
if (a >= _dx)
{
    a = a - _dx - _dx;
    y = y + 1;
}
```

下面是新的绘制结果——



这里一定要清晰地定义边界！ 其实不改变顺序，仅仅通过将 $a \leq 0$ 改为 $a < 0$ 也可以从视觉上解决这种像素突出，但是并没有本质上解决这个问题。这里问题的出现就是我们没有用原始的 a 值来选择像素，而是中间插入了其他操作，而这个操作可能会将 a 从正数变为 0，所以解决的办法也只有这一个。